Database VM Migration

SQL Server, Oracle & PostgreSQL Best Practices on KVM

Database VMs are the highest-risk, highest-reward migration targets. They require careful handling — NUMA-aware placement, hugepages, I/O tuning, and application-consistent backups. This guide covers migration best practices for SQL Server, Oracle, PostgreSQL, and MySQL, including licensing optimizations that save $100K+ per year.

SQL Server | Oracle | PostgreSQL | MySQL | NUMA Tuning | I/O Optimization | License Savings

Database Migration Strategy

Databases are migrated last (Wave 2-3) — they're the most critical and require the most tuning.

Database-Specific Migration Considerations

Database	OS	Downtime Window	Key Risk	KVM Optimization	Licensing Impact
SQL Server	Windows	2-4 hours	VirtIO driver injection, activation	CPU pin, hugepages, VirtIO SCSI	Per-core: pin to fewer cores = save $$$
Oracle	Linux/Win	1-2 hours	Oracle license audit on new platform	CPU pin, NUMA, raw/LVM for ASM	Hard partition: pin vCPUs to limit licensing
PostgreSQL	Linux	30 min	initramfs/fstab (standard Linux fixes)	hugepages, I/O scheduler, shared_buffers	No licensing (open source)
MySQL/MariaDB	Linux	30 min	Standard Linux fixes	innodb_buffer_pool, O_DIRECT, AIO	No licensing (open source)
MongoDB	Linux	30 min	WiredTiger cache sizing	Transparent hugepages disabled!	SSPL/Community: no license cost
SQL Server on Linux	Linux	30 min	mssql-server service restart	Same as Linux + SQL Server tuning	Per-core savings with CPU pinning

Migration Sequence (Database Tier)

Pre-Migration (Week Before)

- Full backup of all databases (tested restore)
- Document connection strings, ports, listeners
- Performance baseline: TPS, latency P99, IOPS
- Identify replication topology (primary → replica)
- Notify application teams of maintenance window
- Prepare rollback plan (VMware VM stays powered off)

Migration Day

- **T-2h:** Final incremental backup
- **T-1h:** Stop application connections
- **T-0:** Clean shutdown of database VM
- **T+5m:** Start KVM VM (pre-converted with h2kvmctl)
- **T+10m:** Verify database service started
- **T+15m:** Run integrity checks (DBCC/pg_isready)
- **T+30m:** Reconnect applications, monitor
- **T+2h:** Performance comparison to baseline

SQL Server on KVM

SQL Server runs excellently on KVM — and CPU pinning can save $50K+/yr in licensing.

SQL Server Per-Core Licensing Optimization

The Licensing Opportunity

- SQL Server Standard: $3,945/2-core pack
- SQL Server Enterprise: $15,123/2-core pack
- License cost is based on VM vCPU count (min 4 cores)
- On VMware: VM with 8 vCPUs = 8 core licenses needed
- On KVM: right-size to 4 vCPUs = 4 core licenses needed
- **Enterprise savings: $15,123/yr per VM (4 cores saved)**
- CPU pin guarantees Microsoft recognizes the core limit
- Microsoft accepts KVM CPU pinning for licensing purposes

KVM Configuration for SQL Server

```
<!-- SQL Server optimized domain XML -->
<vcpu placement='static'>4</vcpu>
<cputune>
  <vcpupin vcpu='0' cpuset='4'/>
  <vcpupin vcpu='1' cpuset='5'/>
  <vcpupin vcpu='2' cpuset='6'/>
  <vcpupin vcpu='3' cpuset='7'/>
  <emulatorpin cpuset='0-1'/>
</cputune>
<numatune>
  <memory mode='strict' nodeset='0'/>
</numatune>
<memoryBacking>
  <hugepages/>
  <locked/>
</memoryBacking>
```

SQL Server Migration Checklist

#	Check	Command	Notes
1	VirtIO drivers injected	`h2kvmctl --virtio-drivers`	Prevents BSOD on boot
2	SQL Server service starts	`Get-Service MSSQLSERVER`	Should auto-start after VM boot
3	Database integrity	`DBCC CHECKDB`	Run on every database post-migration
4	Connection test	`sqlcmd -S localhost -Q "SELECT 1"`	Verify SQL Server responding
5	Performance baseline	`sys.dm_os_wait_stats`	Compare wait stats to VMware baseline
6	Tempdb on separate VirtIO disk	Attach second QCOW2 for tempdb	Separate I/O path improves performance
7	Max server memory set	`sp_configure 'max server memory'`	Leave 2GB for OS, rest for SQL Server

8	Windows activation	`slmgr /ato`	Re-activate if hardware ID changed

Oracle on KVM

Oracle licensing on KVM requires careful CPU pinning — "hard partitioning" saves millions.

Oracle Licensing on KVM (Hard Partitioning)

Oracle's Licensing Policy

- Oracle recognizes KVM CPU pinning as "hard partitioning"
- With hard partitioning: license only the pinned vCPUs
- Without pinning: license ALL physical cores on the host!
- Oracle Database Enterprise: $47,500/core
- 64-core host without pinning: $47,500 x 64 x 0.5 = $1,520,000
- Same host with VM pinned to 4 cores: $47,500 x 4 x 0.5 = $95,000
- **Savings: $1,425,000 by CPU pinning!**
- Document pinning in libvirt XML for audit evidence

Oracle KVM Configuration

```
# CPU pinning (CRITICAL for Oracle licensing)
<vcpu placement='static'>4</vcpu>
<cputune>
  <vcpupin vcpu='0' cpuset='8'/>
  <vcpupin vcpu='1' cpuset='9'/>
  <vcpupin vcpu='2' cpuset='10'/>
  <vcpupin vcpu='3' cpuset='11'/>
</cputune>

# Oracle licensing core factor:
# Intel Xeon: 0.5 (4 vCPUs = 2 Oracle licenses)
# AMD EPYC: 0.5 (4 vCPUs = 2 Oracle licenses)
# Cost: 2 x $47,500 = $95,000
# vs unpinned: 32 x $47,500 = $1,520,000
```

Oracle ASM on KVM

```
# Oracle ASM prefers raw block devices over filesystem
# Option 1: LVM logical volumes presented as raw disks
lvcreate -L 100G -n ora-data vg0
lvcreate -L 50G -n ora-redo vg0
# In libvirt XML:
# <disk type='block' device='disk'>
#   <source dev='/dev/vg0/ora-data'/>
#   <target dev='vdb' bus='virtio'/>
# </disk>

# Option 2: QCOW2 with cache=none, io=native (almost as fast)
# <driver name='qemu' type='qcow2' cache='none' io='native' discard='unmap'/>
```

```
# Performance tip: separate QCOW2/LVs for data, redo, temp, archive
# Each on different NVMe for parallel I/O
```

PostgreSQL & MySQL on KVM

Open-source databases run beautifully on KVM — no licensing concerns, just performance tuning.

PostgreSQL KVM Tuning

postgresql.conf Tuning for KVM

```
# Memory (VM with 32GB RAM)
shared_buffers = 8GB        # 25% of VM RAM
effective_cache_size = 24GB # 75% of VM RAM
work_mem = 256MB
maintenance_work_mem = 2GB

# WAL
wal_buffers = 64MB
checkpoint_completion_target = 0.9
max_wal_size = 4GB

# I/O (VirtIO disk with cache=none)
effective_io_concurrency = 200  # NVMe backend
random_page_cost = 1.1          # NVMe (not spinning disk)

# Hugepages (KVM host provides 2MB pages)
huge_pages = try

# Connections
max_connections = 200
# Use PgBouncer for connection pooling
```

Host-Level Tuning for DB VMs

```
# 1. Hugepages (host level)
echo 4096 > /proc/sys/vm/nr_hugepages
# 4096 x 2MB = 8GB for PostgreSQL shared_buffers

# 2. I/O scheduler (host level)
echo none > /sys/block/nvme0n1/queue/scheduler
# "none" for NVMe (already has internal scheduler)

# 3. Disk cache mode (libvirt)
# cache='none' io='native' — bypasses host page cache
# Best for databases that manage their own cache

# 4. CPU governor
cpupower frequency-set -g performance
# Prevents CPU frequency scaling during queries

# 5. Swappiness (guest level)
echo 1 > /proc/sys/vm/swappiness
# Minimize swap usage for database VM

# 6. THP disabled for some DBs
echo never > /sys/kernel/mm/transparent_hugepages/enabled
# Required for MongoDB, optional for PostgreSQL
```

Database Performance: VMware vs KVM

Benchmark	VMware ESXi	KVM (default)	KVM (tuned)	Improvement	KVM Tuning Applied
pgbench TPS	12,500	11,800	**14,200**	**+14%**	Hugepages + CPU pin + cache=none
sysbench OLTP	8,200 TPS	7,900	**9,100**	**+11%**	NUMA + hugepages + io=native
fio 4K IOPS	85,000	82,000	**105,000**	**+24%**	VirtIO + cache=none + io_uring

fio latency P99	0.18ms	0.20ms	**0.12ms**	**-33%**	CPU pin + hugepages + polling

Database VMs on KVM: Better Performance + Massive License Savings

Tuned KVM outperforms VMware ESXi for database workloads thanks to hugepages, NUMA-aware placement, and VirtIO's lower overhead. SQL Server saves $15K+/VM/yr with right-sized CPU pinning. Oracle saves $1M+ with hard-partitioned KVM (vs full-host licensing). PostgreSQL and MySQL benefit from near-native NVMe performance. h2kvmctl migrates database VMs with all fixes applied — initramfs, fstab, VirtIO — and generates NUMA-aware domain XML.